

Dispositivos Conectados

Protocolos IP para IoT: Modelo orientado ao *stream*

Modelo de Comunicação Orientado ao *Stream*



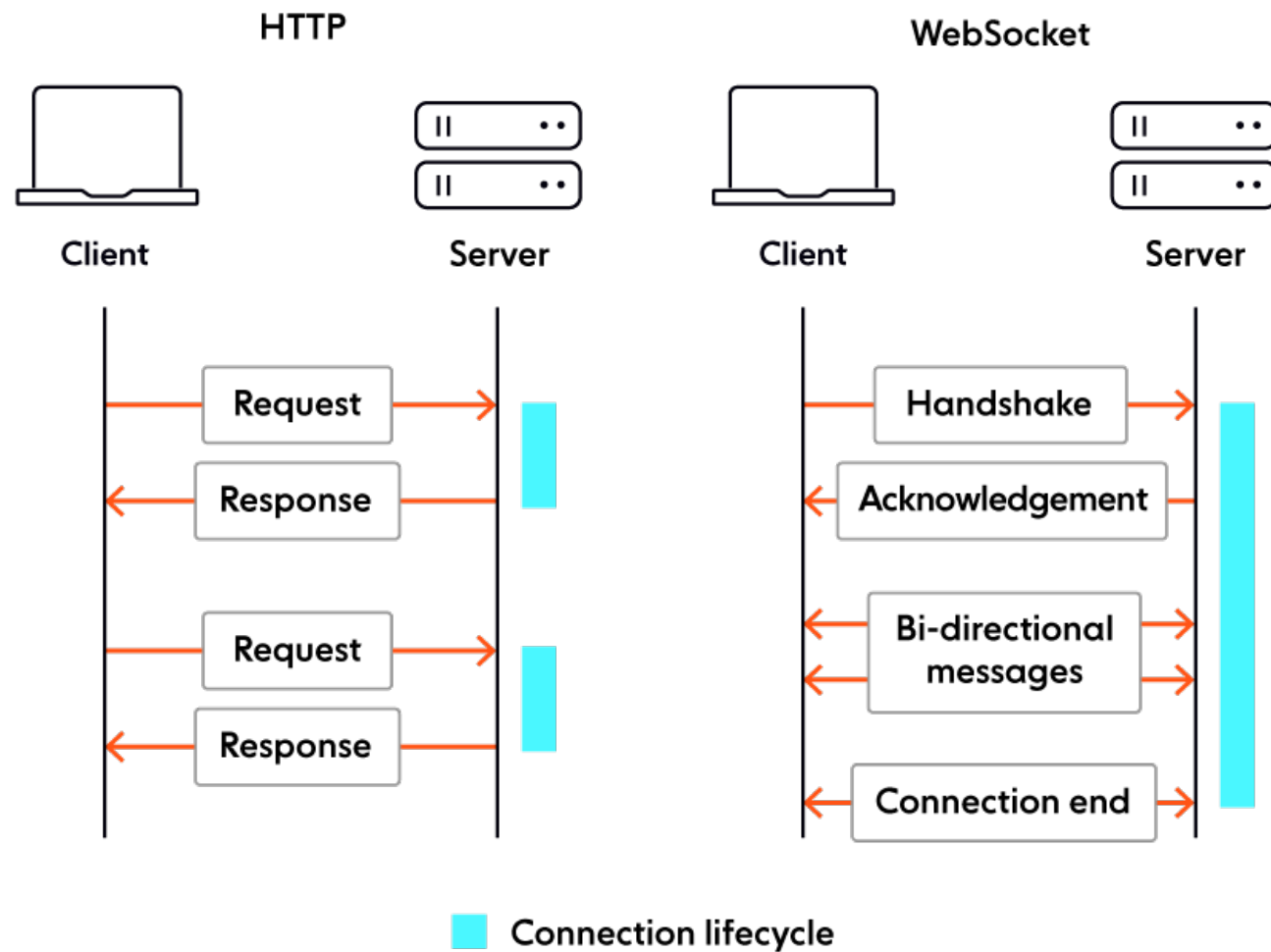
- A comunicação é estabelecida através de uma **ligação contínua entre emissor e recetor**.
- **Dados fluem como um fluxo de bytes** (*stream*), mantendo o estado da ligação durante toda a sessão de comunicação.
- Uma das vantagens deste modelo é a **garantia de entrega e de ordem dos dados**, assegurada por mecanismos de controlo presentes em protocolos como o TCP
 - Adequado para aplicações que exigem **baixa latência e sincronização precisa**, como o *streaming* de vídeo e áudio ou a recolha contínua de dados de sensores.
- Uma das desvantagens é um **overhead superior**, uma vez que a manutenção da sessão e o controlo de fluxo consomem mais memória e energia
 - Relevante em dispositivos IoT com recursos limitados.
- A **segurança integrada** é uma componente essencial: o uso de TLS/DTLS assegura a confidencialidade e integridade do fluxo de dados ao longo de toda a comunicação.
- Os **WebSockets são um protocolo de comunicação orientado ao stream** que permite uma ligação TCP persistente e bidirecional entre cliente e servidor.
 - Permitem que ambas as partes enviem dados a qualquer momento, sem necessidade de iniciar novas ligações.

A breve história dos WebSockets

- Antes dos WebSockets, a comunicação em tempo real na web era realizada com **técnicas que criavam várias ligações HTTP sucessivas** (e.g., *long polling*, Comet e AJAX), aumentando a latência e o consumo de recursos.
- Por volta de 2008–2009, Michael Carter e Ian Hickson (no âmbito do HTML5) propuseram um novo protocolo que permitia a **comunicação bidirecional e contínua entre navegador e servidor**.
- O protocolo foi desenvolvido e posteriormente normalizado pelo IETF, resultando na publicação do **RFC 6455 – “The WebSocket Protocol”** em dezembro de 2011.
- Os principais *browsers* (Chrome, Firefox, Safari, Edge e Internet Explorer 10) adotaram o **suporte nativo ao WebSocket entre 2010 e 2012**.
- No lado do servidor, surgiram bibliotecas como Socket.IO, SignalR e ws (Node.js) que facilitaram a integração do protocolo em aplicações web.
- Atualmente, os WebSockets são **amplamente usados em aplicações web e IoT em tempo-real**, incluindo *dashboards*, jogos online, monitorização de sensores, entre outros.
- Apesar da sua popularidade, novos protocolos como HTTP/2, HTTP/3 (QUIC) e *WebRTC Data Channels* oferecem alternativas mais modernas e eficientes para comunicações de baixa latência e *peer-to-peer*.



WebSockets vs. Modelo tradicional de pedido/resposta HTTP



Aplicações e Vantagens do uso de WebSockets

As **utilizações dos WebSockets** podem ser agrupadas em duas categorias:

- **Atualizações em tempo-real**: onde a comunicação é unidirecional e o servidor transmite ao cliente atualizações de baixa latência (e frequentemente muito frequentes).
 - Exemplos: alertas e *dashboards* em tempo-real.
- **Comunicação bidirecional**: onde tanto o cliente como o servidor podem enviar e receber mensagens.
 - Exemplos: aplicações de chat, eventos virtuais e salas de aula virtuais. Os WebSockets também podem servir de base para funcionalidades de colaboração síncrona entre vários utilizadores, como a edição simultânea de um mesmo documento.

Principais vantagens (comparação com o HTTP) :

- **Melhor desempenho**: eliminam a necessidade de criar uma nova ligação a cada pedido, reduzindo drasticamente o tamanho de cada mensagem (por não incluir cabeçalhos HTTP).
 - Isto poupa largura de banda, melhora a latência e torna os WebSockets mais escaláveis do que o HTTP.
- **Extensibilidade**: a flexibilidade faz parte do próprio design da tecnologia WebSocket, permitindo a implementação de subprotocolos (protocolos de nível de aplicação) e extensões para funcionalidades adicionais.
- **Tempo de reação rápido**: sendo uma tecnologia orientada a eventos, os WebSockets permitem transferir dados sem que o cliente precise de os solicitar.
 - Esta característica é especialmente útil em cenários onde o cliente precisa reagir rapidamente a um evento inesperado, como um alerta.



Protocolo WebSocket

- O protocolo WebSocket permite uma **comunicação contínua, full-duplex e bidirecional** entre servidores e clientes web, utilizando uma ligação TCP subjacente.
- O funcionamento base inicia-se com **um handshake de abertura**, no qual a ligação é atualizada de HTTP para WebSocket.
- Após a negociação bem-sucedida do *handshake*, estabelece-se um **canal de comunicação persistente**, no qual ambas as partes podem enviar dados de forma independente e a qualquer momento.
- Os **dados são trocados em mensagens**, que podem ser compostas por um ou vários frames (fragmentos de dados).
- Quando a **comunicação termina, a ligação é encerrada através de um handshake de fecho**, concluindo a sessão WebSocket.

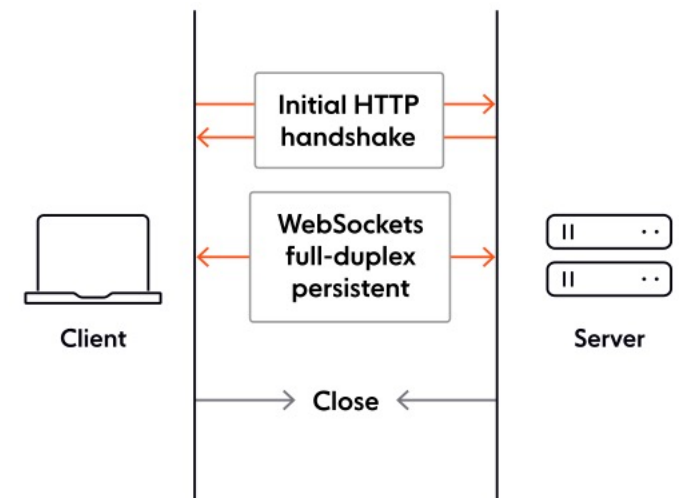
Internet Engineering Task Force (IETF)
Request for Comments: 6455
Category: Standards Track
ISSN: 2070-1721

I. Fette
Google, Inc.
A. Melnikov
Isode Ltd.
December 2011

The WebSocket Protocol

Abstract

The WebSocket Protocol enables two-way communication between a client running untrusted code in a controlled environment to a remote host that has opted-in to communications from that code. The security model used for this is the origin-based security model commonly used by web browsers. The protocol consists of an opening handshake followed by basic message framing, layered over TCP. The goal of this technology is to provide a mechanism for browser-based applications that need two-way communication with servers that does not rely on opening multiple HTTP connections (e.g., using XMLHttpRequest or <iframe>s and long polling).



Esquema e sintaxe do WebSocket URI

- O protocolo WebSocket define **dois esquemas de URI** para o tráfego entre servidor e cliente:
 - **ws**: utilizado para ligações não encriptadas.
 - **wss**: utilizado para ligações seguras e encriptadas através de Transport Layer Security (TLS).
- Os esquemas de URI do WebSocket são análogos aos do HTTP: o esquema **wss** utiliza o mesmo mecanismo de segurança que o https para proteger as ligações, enquanto **ws** corresponde ao http.
- O restante formato do URI do WebSocket segue uma **sintaxe genérica, semelhante à do HTTP**, composta por vários elementos: host, porta, caminho (path) e parâmetros de consulta (query), conforme ilustrado no seguinte exemplo abaixo:

`wss://example.com:443/websocket/demo?foo=bar`

Scheme Host Port Path Query

Handshake de abertura

- O processo de estabelecimento de uma ligação WebSocket é conhecido como handshake de abertura e consiste numa troca de pedido/resposta HTTP/1.1 entre o cliente e o servidor.
 - O cliente é sempre quem inicia o handshake, enviando um pedido GET ao servidor para indicar que pretende atualizar a ligação de HTTP para WebSocket.
- O servidor deve responder com o código HTTP 101 Switching Protocols para que a ligação WebSocket seja estabelecida. A partir desse momento, a ligação passa a permitir comunicação contínua, bidirecional e full-duplex entre o servidor e o cliente..

Pedido do cliente

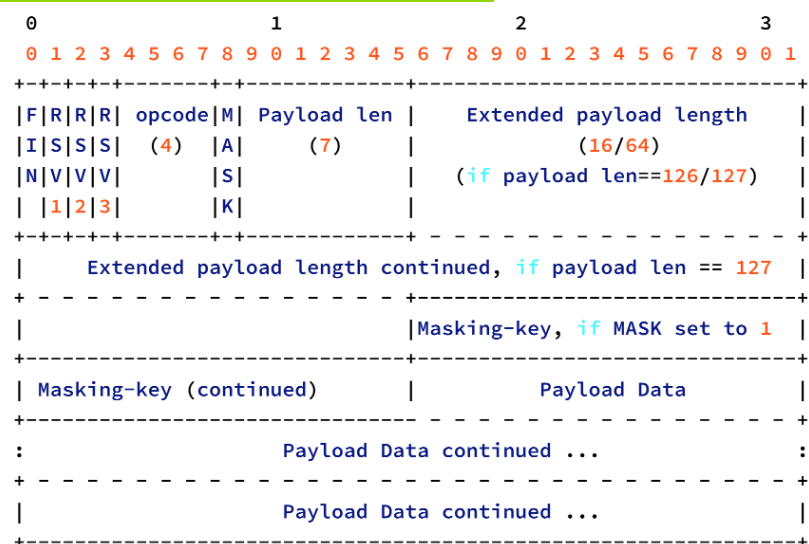
```
GET wss://example.com:8181/ HTTP/1.1
Host: localhost: 8181
Connection: Upgrade
Upgrade: websocket
Sec-WebSocket-Version: 13
Sec-WebSocket-Key: zy6Dy9mSAIM7GJZNf9rI1A==
```

Resposta do servidor

```
HTTP/1.1 101 Switching Protocols
Connection: Upgrade
Sec-WebSocket-Accept: EDJa7WCAQQzMCYNJM42Syuo9SqQ=
Upgrade: websocket
```


Trama WebSocket

- Após um *handshake* de abertura bem-sucedido, o cliente e o servidor podem utilizar a ligação WebSocket para trocar **mensagens em modo full-duplex**.
- Uma mensagem WebSocket é **composta por uma ou mais tramas**.
- A trama WebSocket possui uma **sintaxe binária** e contém vários elementos de informação:
 - **Bit FIN**: indica se a trama corresponde ao fragmento final de uma mensagem WebSocket. Se for, o bit FIN deve ser definido como 1. Todos as outras tramas devem ter o bit FIN a 0.
 - **RSV 1, 2, 3**: reservados para extensões do WebSocket.
 - **Opcode**: determina como interpretar os dados do *payload*.
 - **Mask**: indica se o *payload* está mascarado ou não.
 - **Masking key**: chave usada para desmascarar os dados do *payload*.
 - **Comprimento do payload**: indica o tamanho do *payload*.
 - **Dados do payload**: contêm os dados da aplicação e das extensões.



Opcodes WebSocket

OPCODE	DESCRIPTION
0	Continuation frame; continues the payload from the previous frame.
1	Indicates a text frame (UTF-8 text data).
2	Indicates a binary frame.
3-7	Reserved for custom data frames.
8	Connection close frame; leads to the connection being terminated.
9	A ping frame. Serves as a heartbeat mechanism ensuring the connection is still alive. The receiver must respond with a pong frame.
10	A pong frame. Serves as a heartbeat mechanism ensuring the connection is still alive. Sent as a response after receiving a ping frame.
11-15	Reserved for custom control frames.

Cada trama possui um **opcode que determina como deve ser interpretado o conteúdo do respetivo payload**. Os opcodes padrão atualmente em uso são definidos pelo RFC 6455 e mantidos pela IANA.



Handshake de fecho

- O *handshake* de fecho é um processo simples e iniciado com o envio de uma **trama de fecho com opcode igual a 8**.
- Além do opcode, esta trama pode conter um corpo que indica o motivo do encerramento, um **código de estado** (inteiro) e uma **string codificada em UTF-8** que descreve a razão.
- Os códigos de estado padrão que podem ser usados durante o *handshake* de fecho são definidos no **RFC 6455**.

STATUS CODE	NAME	DESCRIPTION
0-999	N/A	Codes below 1000 are invalid and cannot be used.
1000	Normal closure	Indicates a normal closure, meaning that the purpose for which the WebSocket connection was established has been fulfilled.
1001	Going away	Should be used when closing the connection and there is no expectation that a follow-up connection will be attempted (e.g., server shutting down, or browser navigating away from the page).
1002	Protocol error	The endpoint is terminating the connection due to a protocol error.

Subprotocolos (RFC 6455)

- São **protocolos de nível de aplicação** construídos sobre o protocolo WebSocket básico.
- Servem para definir **formatos específicos e semânticas de comunicação de nível superior** entre cliente e servidor.
- Garantem não só o **acordo sobre a forma como os dados são estruturados**, mas também sobre a **forma como a comunicação deve iniciar-se**, decorrer e terminar.
- Podem ser agrupados em **três categorias** principais:
 - **Protocolos registados**: referem-se a protocolos registados junto da IANA.
 - **Protocolos abertos**: incluem protocolos como o Message Queuing Telemetry Transport (MQTT), por exemplo.
 - **Protocolos personalizados**: dizem respeito a bibliotecas *open-source* ou soluções proprietárias que introduzem a sua própria variante de comunicação baseada em WebSockets.
- Os subprotocolos **são negociados durante o handshake de abertura**. O cliente utiliza o cabeçalho *Sec-WebSocket-Protocol* para indicar um ou mais subprotocolos separados por vírgulas, conforme ilustrado:

```
Sec-WebSocket-Protocol: amqp, v12.stomp
```

- Se o servidor reconhecer os subprotocolos enviados no pedido do cliente, deve selecionar um (e apenas um) e devolvê-lo juntamente com o cabeçalho *Sec-WebSocket-Protocol*.

Extensões

- **Estendem o protocolo WebSocket**, podendo ser utilizadas para adicionar novos *opcodes*, campos de dados e funcionalidades adicionais ao protocolo WebSocket padrão.
- Uma **extensão registrada na IANA** é, por exemplo, a *permessage-deflate*, que comprime a parte dos dados do *payload* das tramas WebSocket.
- As extensões são **negociadas durante o handshake de abertura**. O cliente utiliza o cabeçalho `Sec-WebSocket-Extensions` para indicar as extensões que pretende utilizar, conforme ilustrado no exemplo seguinte.

```
Sec-WebSocket-Extensions: permessage-deflate, my-custom-extension
```

- Se o servidor suportar as extensões enviadas no pedido do cliente, deve incluí-las na resposta, juntamente com o cabeçalho `Sec-WebSocket-Extensions`.
- A partir desse momento, o cliente e o servidor podem comunicar através de WebSockets utilizando as extensões negociadas.

Kahoot! Time

